

02 — Heap Storage

Module: PostgreSQL Internals | **Difficulty:** Advanced | **Est. reading time:** 55 min

Table of Contents

1. [Learning Objectives](#)
 2. [Overview](#)
 3. [Heap File Structure](#)
 4. [The 8 KB Page Layout](#)
 5. [PageHeaderData \(24 bytes\)](#)
 6. [ItemIdData Array](#)
 7. [Tuple Layout in the Page](#)
 8. [HeapTupleHeaderData](#)
 9. [NULL Bitmap](#)
 10. [Data Alignment Rules](#)
 11. [MVCC Versions in the Same Page](#)
 12. [ASCII Diagrams](#)
 13. [Live Observation Queries](#)
 14. [Common Mistakes](#)
 15. [Best Practices](#)
 16. [Performance Considerations](#)
 17. [Interview Questions](#)
 18. [Exercises with Solutions](#)
 19. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Draw the layout of an 8 KB PostgreSQL page from memory, labelling every region.
 - Explain what each field in `PageHeaderData` means and how the server uses it.
 - Describe the content of a heap tuple header, including MVCC fields `t_xmin`, `t_xmax`, and `t_ctid`.
 - Explain how the `infomask` bits control visibility and tuple state.
 - Describe how the NULL bitmap works and under what conditions it is omitted.
 - Predict the physical size of a tuple given its column types and alignment requirements.
 - Use `pageinspect` to observe all these structures on a live database.
-

Overview

PostgreSQL stores table data in **heap files** — fixed-size (8 KB by default) pages, laid out sequentially in the relation file. "Heap" does not imply any ordering; it simply means that rows are stored in no particular order. (Clustered tables in PostgreSQL are not truly clustered after the first `CLUSTER` command — subsequent insertions are not ordered.)

Every table access — sequential scan, index scan, bitmap heap scan — ultimately reads one or more 8 KB pages into the buffer pool and finds tuples within them.

Heap File Structure

A heap file is a flat sequence of 8 KB blocks:

Relation file (e.g. base/16384/16385)

Page 0 (8192 B)	Page 1 (8192 B)	Page 2 (8192 B)	...	Page N-1 (8192 B)
block 0	block 1	block 2		block N-1

Page numbers start at 0. The (blockno, itemno) pair forms a **ctid** (physical location pointer), e.g. (0,1) means page 0, item 1.

The default block size is 8192 bytes. It is set at compile time (`BLCKSZ`) and cannot be changed on a running cluster without a dump/restore.

The 8 KB Page Layout

Byte offset	Content
0	PageHeaderData (24 bytes)
24	ItemIdData[1] (4 bytes per item pointer)
28	ItemIdData[2]
...	... (grows downward toward the middle)
	↓ lower free pointer (pd_lower)
	← free space →
	↑ upper free pointer (pd_upper)
...	Tuple data (grows upward from end of page)
	HeapTupleHeaderData + NULL bitmap + data
8192-X	last tuple in page
8192	end of page (pd_special, 8192 for heap tables)

Key pointers in the header:

- **pd_lower** — byte offset to the first free byte after the last `ItemId` .
- **pd_upper** — byte offset to the start of the most recently inserted tuple.
- **pd_special** — byte offset to the "special area" at the end (used by indexes; for heap pages this equals 8192).

Free space in the page = `pd_upper - pd_lower` .

PageHeaderData (24 bytes)

Defined in `src/include/storage/bufpage.h` :

```
typedef struct PageHeaderData {  
    PageXLogRecPtr pd_lsn;          /* 8 bytes: LSN of last WAL record touching page
```

```

*/
uint16      pd_checksum;    /* 2 bytes: page checksum (if enabled) */
uint16      pd_flags;      /* 2 bytes: flag bits (PD_HAS_FREE_LINES etc.) */
LocationIndex pd_lower;    /* 2 bytes: offset to free space start */
LocationIndex pd_upper;    /* 2 bytes: offset to free space end */
LocationIndex pd_special;  /* 2 bytes: offset to special space */
uint16      pd_pagesize_version; /* 2 bytes: page size + layout version */
TransactionId pd_prune_xid; /* 4 bytes: prunable XID hint */
} PageHeaderData;
/* Total: 8+2+2+2+2+2+4 = 24 bytes */

```

Field-by-field

Field	Size	Meaning
pd_lsn	8 B	Log Sequence Number of the last WAL record that modified this page. Used during recovery to skip pages already replayed.
pd_checksum	2 B	Optional page checksum (enabled with <code>pg_checksums</code>). Zero if checksums are off.
pd_flags	2 B	Bit flags: <code>PD_HAS_FREE_LINES</code> (dead line pointers exist), <code>PD_PAGE_FULL</code> (no free space), <code>PD_ALL_VISIBLE</code> (all tuples visible to all transactions).
pd_lower	2 B	Offset in bytes from page start to the end of the <code>ItemId</code> array.
pd_upper	2 B	Offset in bytes from page start to the start of the newest live tuple.
pd_special	2 B	Offset to the special area (8192 for plain heap pages; smaller for index pages that store index-specific data there).
pd_pagesize_version	2 B	Encodes the page size and layout version. Sanity-checked at startup.
pd_prune_xid	4 B	Hint: the oldest <code>t_xmax</code> on this page that might be prunable. Zero means unknown. Used to quickly skip pages that need no pruning.

ItemIdData Array

After the 24-byte header comes an array of 4-byte **line pointers** (`ItemIdData`), one per tuple slot. They grow from byte 24 toward higher addresses.

```

typedef struct ItemIdData {
    unsigned lp_off : 15; /* byte offset to start of tuple (within page) */
    unsigned lp_flags : 2; /* LP_UNUSED=0, LP_NORMAL=1, LP_REDIRECT=2, LP_DEAD=3 */
    unsigned lp_len : 15; /* length of tuple in bytes */
} ItemIdData;

```

lp_flags	Meaning
LP_UNUSED (0)	Slot has never been used or has been recycled.
LP_NORMAL (1)	Active tuple. <code>lp_off</code> and <code>lp_len</code> are valid.

LP_REDIRECT (2)	HOT redirect: lp_off is the ItemId of the updated tuple on the same page.
LP_DEAD (3)	Dead tuple, can be reclaimed by VACUUM.

A **ctid** like (0, 3) means: page 0, ItemId slot 3. The ItemId at slot 3 gives the byte offset within the page where the tuple starts.

Tuple Layout in the Page

A heap tuple physically consists of:

HeapTupleHeaderData (23+ bytes)
t_xmin (4 B)
t_xmax (4 B)
t_cid / t_xvac (4 B)
t_ctid (6 B)
t_infomask2 (2 B)
t_infomask (2 B)
t_hoff (1 B)
NULL bitmap (optional, padded) ceil(natts / 8) bytes
OID (optional, 4 B) [deprecated]
User data (column values) Aligned to type requirements

t_hoff stores the byte offset from the start of the tuple header to the start of user data. It is always a multiple of **MAXALIGN** (8 bytes on 64-bit systems).

HeapTupleHeaderData

```
struct HeapTupleHeaderData {
    union {
        HeapTupleFields t_heap; /* normal tuple */
        DatumTupleFields t_datum; /* composite type datum */
    } t_choice;

    ItemPointerData t_ctid; /* 6 bytes: current TID of this tuple (or newer
version) */
    uint16 t_infomask2; /* 2 bytes: number of attributes + flags */
    uint16 t_infomask; /* 2 bytes: visibility/state flags */
    uint8 t_hoff; /* 1 byte: offset to user data */
    /* bits: t_bits[] optional NULL bitmap */
};
```

Where `HeapTupleFields` is:

```
typedef struct HeapTupleFields {
    TransactionId t_xmin;    /* 4 bytes: inserting transaction XID */
    TransactionId t_xmax;    /* 4 bytes: deleting/updating transaction XID (0 if
live) */
    union {
        CommandId t_cid;    /* 4 bytes: inserting/deleting command within
transaction */
        TransactionId t_xvac; /* used during VACUUM FULL */
    } t_field3;
} HeapTupleFields;
```

MVCC Fields in Detail

Field	Size	Meaning
t_xmin	4 B	Transaction ID that inserted this tuple version. A transaction can see the tuple only if t_xmin is committed and satisfies the snapshot.
t_xmax	4 B	Transaction ID that deleted or updated this tuple version. Zero means the tuple is live (not deleted). If non-zero and committed, the tuple is dead to all new snapshots.
t_cid	4 B	Command ID within t_xmin's transaction. Used for statement-level visibility within the same transaction (you cannot see rows inserted by a later command in the same query).
t_ctid	6 B	Physical location of this tuple (its own ctid normally). When an UPDATE creates a new version, the old tuple's t_ctid is updated to point to the new version. This forms the update chain . The chain terminates when t_ctid points to itself.

infomask Bits (`t_infomask`)

`t_infomask` is a 16-bit field that caches MVCC state to avoid looking up the commit-log (`pg_xact`) every time a tuple is accessed.

Bit mask (hex)	Name	Meaning
0x0001	HEAP_HASNULL	Tuple has at least one NULL column; NULL bitmap is present.
0x0002	HEAP_HASVARWIDTH	Tuple has at least one variable-length column.
0x0004	HEAP_HASEXTERNAL	Tuple has at least one out-of-line (TOAST) value.
0x0008	HEAP_HASOID_OLD	(Deprecated) tuple has OID field.
0x0010	HEAP_XMAX_KEYSHR_LOCK	t_xmax is a key-share lock (not delete).
0x0020	HEAP_COMBOCID	t_cid is a combo CID (complex scenario).
0x0040	HEAP_XMAX_EXCL_LOCK	t_xmax is an exclusive lock.

0x0080	HEAP_XMAX_LOCK_ONLY	t_xmax is a lock, not a delete.
0x0100	HEAP_XMIN_COMMITTED	t_xmin is known committed (cached).
0x0200	HEAP_XMIN_INVALID	t_xmin is known aborted (cached).
0x0400	HEAP_XMAX_COMMITTED	t_xmax is known committed (cached).
0x0800	HEAP_XMAX_INVALID	t_xmax is known aborted/zero (tuple is live).
0x1000	HEAP_XMAX_IS_MULTI	t_xmax is a MultiXactId.
0x2000	HEAP_UPDATED	This tuple is the result of an UPDATE.
0x4000	HEAP_MOVED_OFF	(Legacy) tuple moved off by VACUUM FULL.
0x8000	HEAP_MOVED_IN	(Legacy) tuple moved in by VACUUM FULL.

infomask2 Bits (t_infomask2)

The lower 11 bits store `natts` (number of attributes). The upper 5 bits are flags:

Bit	Name	Meaning
0x0800	HEAP_KEYS_UPDATED	Key columns were updated (affects HOT chains).
0x4000	HEAP_HOT_UPDATED	This tuple was HOT-updated (new version on same page).
0x8000	HEAP_ONLY_TUPLE	This is a HOT tuple (no index entry points to it directly).

NULL Bitmap

If `HEAP_HASNULL` is set in `t_infomask`, a NULL bitmap follows the fixed header. It contains one bit per attribute; bit `i` is 1 if attribute `i` is NOT NULL, 0 if NULL.

```
natts = 5 columns → ceil(5/8) = 1 byte of NULL bitmap
```

Bit layout (little-endian within byte):

Byte 0: bit0=col1, bit1=col2, bit2=col3, bit3=col4, bit4=col5, bits5-7=padding

If all columns are NOT NULL:

Byte 0 = 0b00011111 = 0x1F

The NULL bitmap is **absent entirely** when all columns are non-null (HASNULL not set). This saves 1-N bytes per tuple.

`t_hoff` always points past the NULL bitmap (and any OID field) and is padded to `MAXALIGN` (8 bytes). So even a 5-column table with a 1-byte NULL bitmap will have `t_hoff = 24 + 8 = 32` (not 25), because `t_hoff` is rounded up.

Data Alignment Rules

PostgreSQL requires each column value to be aligned to its type's alignment requirement to allow the CPU to load values without unaligned memory access penalties.

Type category	Alignment	Examples
char, bool, "char"	1 byte	char(1), boolean
int2, smallint	2 bytes	smallint
int4, float4, oid	4 bytes	integer, real, date
int8, float8, timestamp	8 bytes	bigint, double precision, timestamp
Variable-length (text, bytea, varchar)	4 bytes (header)	text, varchar, bytea
Arrays, composites	8 bytes	int[], record types

Column order matters for physical tuple size. Columns are stored in their declaration order; padding bytes are inserted between columns to satisfy alignment.

Example — bad column order:

```
CREATE TABLE bad_order (  
  a boolean,      -- 1 byte  
  -- 7 bytes padding to reach 8-byte boundary  
  b bigint,       -- 8 bytes  
  c boolean,      -- 1 byte  
  -- 7 bytes padding  
  d bigint        -- 8 bytes  
);  
-- Fixed header ~24 B + 1+7+8+1+7+8 = 32 bytes data = 56 bytes per tuple
```

Example — good column order (largest-first):

```
CREATE TABLE good_order (  
  b bigint,       -- 8 bytes (offset 0)  
  d bigint,       -- 8 bytes (offset 8)  
  a boolean,      -- 1 byte  (offset 16)  
  c boolean       -- 1 byte  (offset 17)  
  -- 6 bytes padding at end to reach 24-byte alignment  
);  
-- Fixed header ~24 B + 8+8+1+1+6 = 24 bytes data = 48 bytes per tuple (14% smaller)
```

MVCC Versions in the Same Page

When a row is updated, PostgreSQL inserts a new tuple version. The old and new versions may coexist in the same page (if space allows):

Page 0

ItemId[1] → old tuple: t_xmin=100, t_xmax=200

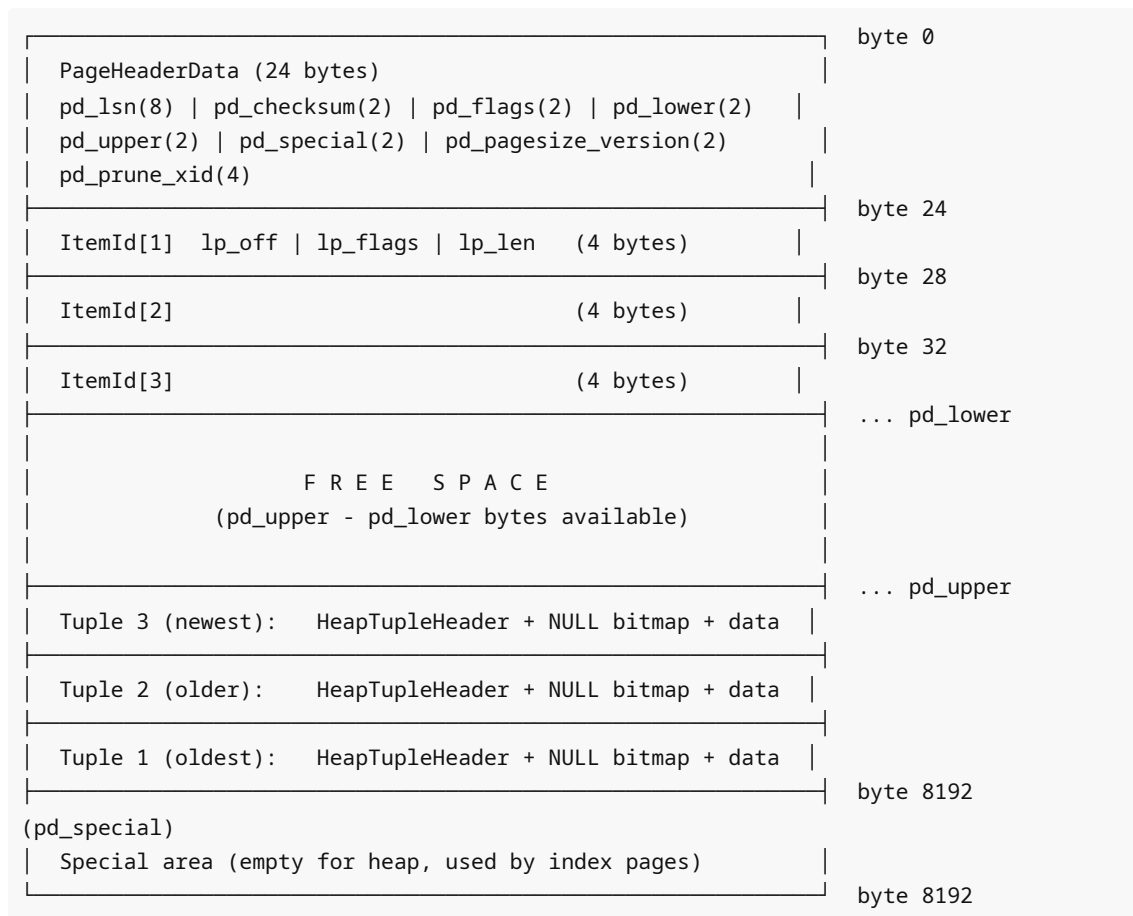
t_ctid = (0,2) ← points to new

ItemId[2] → new tuple: t_xmin=200, t_xmax=0 t_ctid = (0,2) ← points to itself
--

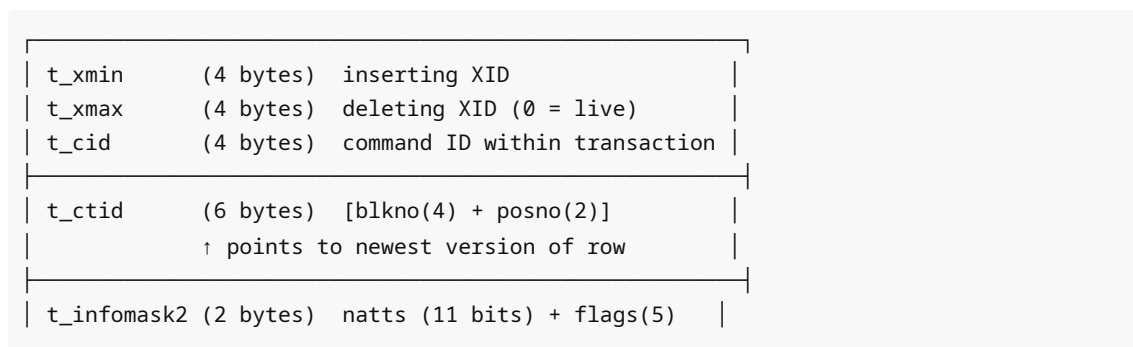
A transaction with snapshot taken before XID 200 sees ItemId[1] (old version). A transaction with snapshot taken after XID 200 commits sees ItemId[2] (new version). Both coexist until VACUUM removes the dead old version.

ASCII Diagrams

Complete 8 KB Page



Heap Tuple Header



t_infomask	(2 bytes)	HASNULL	XMIN_COMMITTED...	
t_hoff	(1 byte)	offset to user data		
				← t_hoff
NULL bitmap (0 or ceil(natts/8) bytes, MAXALIGN-pad)				
Column 1 value (padded to alignment)				
Column 2 value				
...				

Live Observation Queries

Install the `pageinspect` extension (bundled with PostgreSQL, no extra install needed on most distros):

```
-- Enable pageinspect
CREATE EXTENSION IF NOT EXISTS pageinspect;

-- Create a test table
CREATE TABLE heap_demo (
    id      bigint,
    name    text,
    active  boolean
);

INSERT INTO heap_demo VALUES (1, 'Alice', true), (2, 'Bob', false), (3, NULL, NULL);

-- 1. Read raw page header
SELECT * FROM page_header(get_raw_page('heap_demo', 0));
/*
   lsn      | checksum | flags | lower | upper | special | pagesize | version |
prune_xid
-----+-----+-----+-----+-----+-----+-----+-----+
0/16B3488 |         0 |    5 |   56 |  8072 |   8192 |   8192 |      4 |
0
*/

-- 2. List all item pointers on page 0
SELECT * FROM heap_page_items(get_raw_page('heap_demo', 0));
/*
  lp | lp_off | lp_flags | lp_len | t_xmin | t_xmax | t_field3 | t_ctid |
t_infomask2 | t_infomask | t_hoff | t_bits | t_oid | t_data
-----+-----+-----+-----+-----+-----+-----+-----+
1 |  8152 |      1 |    34 |    ... |      0 |      0 | (0,1) |
3 |  2306 |     24 |     |      |      |      |      |
2 |  8112 |      1 |    34 |    ... |      0 |      0 | (0,2) |
3 |  2306 |     24 |     |      |      |      |      |
3 |  8072 |      1 |    36 |    ... |      0 |      0 | (0,3) |
3 |  2305 |     32 |    11 |      |      |      |      |
*/
```

```

-- Note: item 3 has HASNULL set (t_infomask & 1 = 1) and t_hoff=32 (NULL bitmap
present)

-- 3. Observe MVCC fields after an UPDATE
BEGIN;
UPDATE heap_demo SET name = 'Alice Updated' WHERE id = 1;
-- In another session: SELECT * FROM heap_page_items(...) to see old tuple's t_xmax
set

-- 4. Calculate free space in a page
SELECT
    page_number,
    round(100.0 * free_space / 8192, 1) AS free_pct
FROM generate_series(0, (pg_relation_size('heap_demo') / 8192)::int - 1) AS
page_number
CROSS JOIN LATERAL (
    SELECT (
        (page_header(get_raw_page('heap_demo', page_number))).upper -
        (page_header(get_raw_page('heap_demo', page_number))).lower
    ) AS free_space
) sub;

-- 5. Tuple size calculation
SELECT
    relname,
    n_live_tup,
    pg_relation_size(oid) AS total_bytes,
    CASE WHEN n_live_tup > 0
        THEN round(pg_relation_size(oid)::numeric / n_live_tup, 0)
        ELSE NULL END AS avg_bytes_per_tuple
FROM pg_stat_user_tables
WHERE relname = 'heap_demo';

-- 6. Show infomask flags for a tuple (decode manually)
SELECT
    lp,
    t_xmin,
    t_xmax,
    t_ctid,
    t_hoff,
    (t_infomask & 1) > 0 AS has_null,
    (t_infomask & 256) > 0 AS xmin_committed,
    (t_infomask & 512) > 0 AS xmin_invalid,
    (t_infomask & 1024) > 0 AS xmax_committed,
    (t_infomask & 2048) > 0 AS xmax_invalid,
    (t_infomask & 8192) > 0 AS is_updated
FROM heap_page_items(get_raw_page('heap_demo', 0));

```

Common Mistakes

1. **Assuming tuples are stored in insertion order.** They usually are initially, but `VACUUM` and `HOT` updates can leave gaps, and multiple versions of the same row may interleave in the page.
2. **Forgetting that UPDATE leaves a dead tuple.** Every `UPDATE` increases the page's tuple count until `VACUUM` removes the dead old version. High-frequency updates bloat tables rapidly.
3. **Confusing `t_ctid` with a foreign key.** `t_ctid` is a physical pointer to the *next version* of the same logical row, not a relation between tables.
4. **Assuming `t_xmax = 0` means "never updated".** A row with `t_xmax = 0` is currently live. But it may have been updated many times in the past — `t_xmax = 0` on the *current version* just means it hasn't been deleted yet.
5. **Ignoring alignment when estimating tuple size.** Naive column widths summed up will underestimate tuple size because of padding bytes inserted between misaligned columns.
6. **Thinking the NULL bitmap is always present.** It is present only when `HEAP_HASNULL` is set. A 10-column table where all rows have no NULLs saves $\text{ceil}(10/8) = 2$ bytes per tuple by omitting it.
7. **Overlooking `t_hoff` for correct data offset.** Do not assume the header is always 23 bytes; with a NULL bitmap and `MAXALIGN` padding it can be 24, 32, or more.

Best Practices

- **Order columns largest-alignment first** (`bigint` , `double precision` , timestamps) to minimise padding bytes. For wide tables this can reduce tuple size by 5–15%.
 - **Avoid wide sparse tables** (many nullable columns rarely populated) — they waste storage in the NULL bitmap and padding.
 - **Use `pageinspect` in development** to validate your understanding of how a specific table's tuples look on disk.
 - **Monitor bloat** — tables with high `UPDATE` rates need regular vacuuming or `autovacuum` tuning to prevent dead tuple accumulation.
 - **Enable page checksums** (`pg_checksums --enable`) in production to detect silent data corruption.
-

Performance Considerations

Factor	Impact	Action
Dead tuples in pages	Wasted I/O reading dead data	Tune <code>autovacuum</code> or run <code>VACUUM</code>
Poor column alignment	Up to 15% tuple size increase	Reorder columns (requires <code>ALTER TABLE + rewrite</code>)
Large <code>t_hoff</code> (many nullable cols)	Extra bytes per tuple	Consolidate nullable into <code>JSONB</code> or split into subtable
Full pages (<code>pd_flags = PAGE_FULL</code>)	<code>INSERTs</code> go to new pages, leaving gaps	Lower <code>FILLFACTOR</code> to reserve space for <code>HOT</code> updates
Page checksums enabled	~1–2% CPU overhead	Worth it in production for data integrity

8 KB vs 32 KB block size	Larger blocks = fewer I/Os for large sequential scans	Must choose at initdb; 8 KB is optimal for OLTP
--------------------------	---	---

Interview Questions

Q1. How large is an 8 KB PostgreSQL page header and what does it contain?

24 bytes. It contains: `pd_lsn` (8 B, WAL LSN), `pd_checksum` (2 B), `pd_flags` (2 B), `pd_lower` (2 B, end of `ItemId` array), `pd_upper` (2 B, start of tuples from end), `pd_special` (2 B), `pd_pagesize_version` (2 B), and `pd_prune_xid` (4 B, pruning hint).

Q2. What is a ctid and what format does it take?

A `ctid` (`ItemPointerData`) is a 6-byte physical tuple location: a 4-byte block number and a 2-byte offset number (item slot). For example `(0, 1)` means page 0, item slot 1. The server uses `ctids` internally for HOT update chains and heap fetches from index scans.

Q3. What are `t_xmin` and `t_xmax` used for?

MVCC: `t_xmin` is the `XID` of the transaction that inserted this tuple version. `t_xmax` is the `XID` of the transaction that deleted or updated it (0 = still live). A transaction with a snapshot decides visibility by comparing its snapshot to these `XIDs`.

Q4. When is the NULL bitmap present in a tuple?

Only when the `HEAP_HASNULL` bit is set in `t_infomask`, which happens when at least one column in the tuple is NULL. If all columns are non-null the bitmap is omitted entirely to save space.

Q5. Why does `t_hoff` need to be a multiple of `MAXALIGN` ?

The user data starts at offset `t_hoff` from the tuple start. The first column may require 8-byte alignment (e.g., `bigint`). If `t_hoff` were not a multiple of 8, the first column would be misaligned, causing either a CPU fault (on strict architectures) or a performance penalty.

Q6. What does `t_ctid` pointing to a different page/slot mean?

It means this tuple version has been superseded by an `UPDATE`. The new version is at the location `t_ctid` points to. `VACUUM` uses this chain to find the latest live version and remove dead predecessors.

Q7. How does `HEAP_XMAX_INVALID` speed up tuple visibility checks?

If `HEAP_XMAX_INVALID` is set in `t_infomask`, it means `t_xmax` is either zero or from an aborted transaction, so the tuple is definitely live. The server can skip the expensive `pg_xact` lookup entirely.

Q8. What is the minimum possible size of a heap tuple?

Fixed header = 23 bytes + possible NULL bitmap + `MAXALIGN` padding. For a zero-column table (hypothetically) the minimum header is 24 bytes (after rounding `t_hoff` up). In practice the minimum is the header size for whatever columns you declare, typically 24–32 bytes before any data.

Q9. How does column ordering affect physical tuple size?

Padding bytes are inserted between columns to meet alignment requirements. If a 1-byte `boolean` precedes an 8-byte `bigint`, 7 bytes of padding are inserted. Declaring 8-byte columns first, then 4-byte, then 2-byte, then 1-byte eliminates all inter-column padding.

Q10. What extension lets you inspect raw page contents?

`pageinspect` (bundled with PostgreSQL). Key functions: `page_header()` for the 24-byte header, `heap_page_items()` for ItemId and tuple headers, `get_raw_page()` to read the raw bytes.

Exercises with Solutions

Exercise 1 — Compute expected tuple size

Task: For the table below, compute the expected physical tuple size (header + data, no NULLs):

```
CREATE TABLE t1 (  
  a boolean,    -- 1 byte, align 1  
  b integer,    -- 4 bytes, align 4  
  c bigint,     -- 8 bytes, align 8  
  d text        -- variable, align 4  
);  
INSERT INTO t1 VALUES (true, 42, 9999999, 'hello');
```

Solution:

- Header: 23 bytes → round up to `MAXALIGN(8)` → 24 bytes (`t_hoff = 24` , no NULLs)
- Column layout from offset 24:
 - offset 24: `a` (boolean, 1 byte) → end at 25
 - padding: 3 bytes to reach offset 28 (align 4 for `b`)
 - offset 28: `b` (integer, 4 bytes) → end at 32
 - padding: 0 bytes (32 is already 8-byte aligned for `c`)
 - offset 32: `c` (bigint, 8 bytes) → end at 40
 - offset 40: `d` (text 'hello'): 4-byte varlena header + 5 bytes = 9 bytes → end at 49
 - padding: 7 bytes to reach next `MAXALIGN(8)` boundary → tuple end at 56
- **Total: 56 bytes**

Verify:

```
CREATE EXTENSION pageinspect;  
SELECT lp_len FROM heap_page_items(get_raw_page('t1', 0)) LIMIT 1;  
-- Should be 56
```

Exercise 2 — Observe dead tuples

Task: Create a table, perform 100 updates on a single row, and observe dead tuples with `pageinspect` .

```
-- Setup  
CREATE TABLE mvcc_test (id int, val text);  
INSERT INTO mvcc_test VALUES (1, 'version_0');  
  
-- Run 100 updates  
DO $$  
BEGIN  
  FOR i IN 1..100 LOOP
```

```

        UPDATE mvcc_test SET val = 'version_' || i WHERE id = 1;
    END LOOP;
END;
$$;

-- Check dead tuples
SELECT lp, lp_flags, t_xmin, t_xmax, t_ctid
FROM heap_page_items(get_raw_page('mvcc_test', 0))
ORDER BY lp;
-- Many rows with lp_flags=1 and t_xmax != 0 (dead) + one live tuple

-- Check pg_stat_user_tables
SELECT relname, n_live_tup, n_dead_tup
FROM pg_stat_user_tables WHERE relname = 'mvcc_test';

-- Vacuum it
VACUUM mvcc_test;

-- Dead tuples should drop to 0; lp_flags should show LP_DEAD/LP_UNUSED

```

Exercise 3 — Column reordering

Task: Create two tables with the same columns but different declaration order. Compare their average tuple sizes.

```

CREATE TABLE col_order_bad (
    flag1 boolean,
    val1 bigint,
    flag2 boolean,
    val2 bigint
);

CREATE TABLE col_order_good (
    val1 bigint,
    val2 bigint,
    flag1 boolean,
    flag2 boolean
);

INSERT INTO col_order_bad
SELECT (i%2=0), i::bigint, (i%3=0), (i*2)::bigint
FROM generate_series(1,1000) i;

INSERT INTO col_order_good
SELECT i::bigint, (i*2)::bigint, (i%2=0), (i%3=0)
FROM generate_series(1,1000) i;

-- Compare sizes
SELECT
    'bad' AS design,
    pg_relation_size('col_order_bad') AS bytes,

```

```
round(pg_relation_size('col_order_bad')::numeric / 1000, 1) AS bytes_per_row
UNION ALL
SELECT
    'good',
    pg_relation_size('col_order_good'),
    round(pg_relation_size('col_order_good')::numeric / 1000, 1);
-- Expected: good is ~14-18% smaller
```

Cross-References

- **01_storage_architecture.md** — How heap files are named and located in `$PGDATA`
- **03_toast.md** — How columns larger than ~2 kB are stored outside the main tuple
- **06_vacuum.md** — How VACUUM identifies and removes dead tuples from pages
- **08_visibility_map.md** — The VM bit set when all tuples on a page are visible
- **09_free_space_map.md** — How free space in pages is tracked and reused
- **11_query_execution_pipeline.md** — How the executor reads tuples from pages